

Text2CSG: Generating CAD models from natural language via constructive solid geometry[☆]

Luo Zhang^a, Gaochao Song^b, Haocong Rao^a, Zhengyu Wen^a, Jiangbei Hu^{c,*,*}, Ying He^a

^a College of Computing and Data Science, Nanyang Technological University, Singapore

^b School of Computing and Data Science, The University of Hong Kong, China

^c School of Software, Dalian University of Technology, China

ARTICLE INFO

Keywords:

Constructive solid geometry
CAD models
3D generation
Large language models

ABSTRACT

Computer-aided design (CAD) plays a central role in engineering, manufacturing, and 3D content creation, but producing precise CAD models directly from natural language is still difficult. We propose Text2CSG, a two-stage framework for producing high-quality CAD models from text. Our approach uses Constructive Solid Geometry (CSG) as an intermediate representation to capture the structured and hierarchical nature of CAD construction. Unlike prior approaches based on sketch-extrusion sequences, Text2CSG exploits the Boolean compositional logic of CSG trees to represent complex shapes. The system consists of two primary components: (1) a geometry decoder that reconstructs 3D shapes from CSG programs, and (2) a structure-aware Transformer conditioned on high-level semantics extracted from a frozen large language model (LLM). By decoupling semantic understanding from geometric generation, our approach benefits from pre-trained LLMs without requiring fine-tuning. Experiments on a newly constructed CSG dataset demonstrate that Text2CSG achieves higher geometric accuracy and stronger structural consistency than existing methods. Furthermore, the utilization of structured CSG programs makes the generation process interpretable and editable, enabling flexible text-driven CAD design. We will release the source code and dataset.

1. Introduction

Computer-aided design (CAD) has long been part of engineering and manufacturing [1], and is routinely used in areas such as mechanical engineering [2,3], architecture [4,5], electronics [6], and aerospace [7], to name just a few. Among common modeling methods, Constructive Solid Geometry (CSG) [8] is a widely used representation that combines basic geometric shapes, such as cubes, spheres, and cylinders, using Boolean operations. The resulting model is recorded as a tree that reflects the sequence of constructive steps, which makes the representation easy to inspect and modify. Its constructive nature has made it a widely used representation in CAD modeling, parametric design, and procedural generation [9], and recent works have incorporated CSG into neural frameworks [10,11] to learn programmatic construction logic.

Recent work in AI has investigated how 3D shapes can be automatically produced from data or descriptions [12]. However, these approaches often struggle to reproduce the construction logic underlying many CAD models. A key reason is that commonly used representations, such as meshes and voxels, encode geometry at a low

level, as seen in methods including MeshGPT [12], DeepCAD [13], Text2CAD [14], and CADmium [15]. A separate line of work attempts to tackle the problem by training large language models (LLMs) specifically for CAD-related tasks [16–20], but such training is computationally demanding and difficult to scale.

Existing CAD generation models typically adopt sequential representations of sketch-and-extrusion pipelines, where modeling operations are linearized into ordered sequences for learning. While modern CAD systems internally maintain expressive dependency graphs (e.g., DAG-based feature representations), such structural information is largely flattened in prior learning-based methods, limiting their ability to explicitly capture hierarchical and compositional relationships. This linearization can lead to difficulties in modeling multi-component structures and complex Boolean interactions, as long-range dependencies and structural constraints must be implicitly inferred from the sequence. In contrast, CSG represents geometry through a constructive hierarchy and allows explicit modeling of structural relationships within a sequence framework, making it a natural choice for modeling

[☆] This article is part of a Special issue entitled: 'AI+Design' published in Computer-Aided Design.

* Corresponding author.

E-mail address: jbhu@dlut.edu.cn (J. Hu).

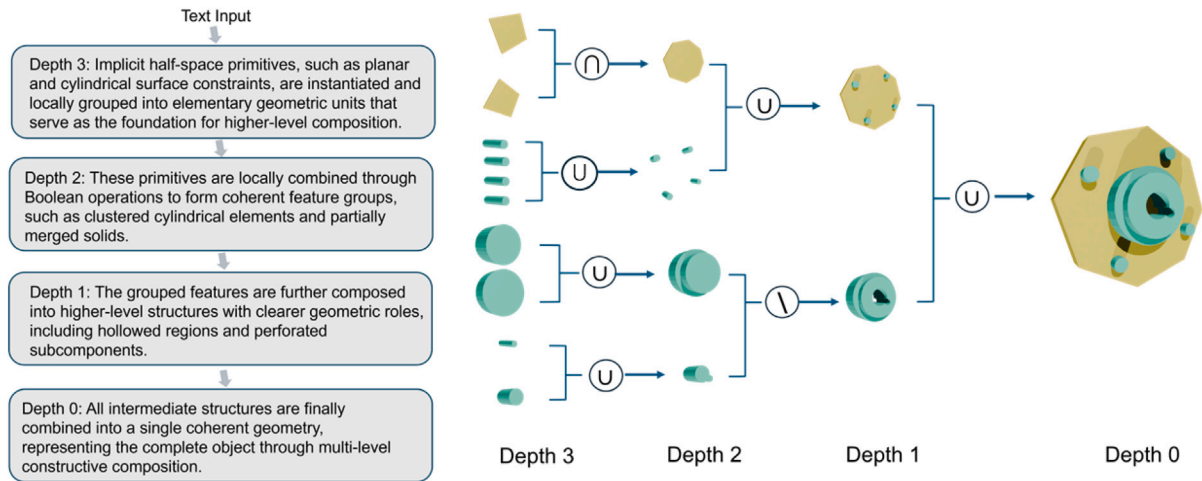


Fig. 1. Text2CSG interprets natural language inputs as structured intent for parametric CSG generation, supporting both conceptual and fully specified descriptions.

multi-part assemblies and diverse volumetric relationships. This structural alignment motivates a CSG-based approach for text-driven CAD generation.

In this work, we present *Text2CSG*, a framework that generates CAD models through CSG programs from natural-language descriptions that remain both syntactically valid and structurally meaningful, as illustrated in Fig. 1. The model incorporates mechanisms for handling hierarchical structure, as well as a compact token vocabulary that keeps program generation efficient. Text2CSG consists of two stages. In the first stage, a Transformer decoder is trained on large-scale symbolic CSG data, using hierarchical token embeddings and a specific CSG Validity Loss to ensure syntactic correctness and constructive coherence. By using the hierarchical and compositional features of CSG, our method models construction logic directly, rather than focusing only on visual appearance. With a small set of primitives and Boolean operators, CSG provides a clear yet effective way to represent complex shapes. Compared to sketch-and-extrusion methods, CSG offers a stronger structural basis that fits well with geometric design logic, as shown in Fig. 2. However, since Transformers process linear sequences, creating tree-structured CSG programs is challenging. To address this, we introduce a structure-aware encoding scheme and structured loss functions to help the model learn meaningful construction patterns. In the second stage, we align the structural knowledge acquired from symbolic CSG training with high-level semantic input. During training, both the CSG sequence and textual embeddings are input to the decoder, where cross-attention layers adjust the decoding process at each step. During inference, the model takes only the natural language input and generates the corresponding CSG construction sequence one step at a time. This design maintains structural principles while allowing flexible, meaning-based CAD generation.

Our contributions are twofold:

- We propose a Transformer-based decoder with hierarchical supervision to model the structural composition of CSG programs, which improves generation accuracy for complex CAD models.
- We introduce a CSG validity loss and a tree-structured decoding strategy to significantly reduce invalid generations. Our model effectively produces multi-component CSG programs with low rates of invalidity.

2. Related work

Neural rendering methods, notably Neural Radiance Fields (NeRF) [21] and 3D Gaussian Splatting (3DGS) [22], have driven rapid progress in 3D reconstruction [23–27]. Concurrently, large language

models (LLMs) have accelerated progress in multimodal 3D generation [28–34]. In the fields of engineering, however, CAD models exhibit strict geometric regularities and explicit structural logic. The necessity for high precision and geometric fidelity significantly increases the complexity of 3D reconstruction and generation tasks for CAD models. Numerous studies have emerged to address these challenges.

LLM-based methods. LLMs-based CAD generation has emerged as a new paradigm for interacting with CAD systems [17,19,35–39]. These methods explore mappings from multimodal inputs to structured CAD representations, such as text, point clouds, meshes, multi-view images, and programming codes. CADGPT [39] directly parses text into executable commands for CAD software while CAD-MLLM [19] further expands to multimodal inputs—including images, point clouds, and text. Text-to-CAD [35] introduces a visual-feedback loop in LLMs to iteratively refine CAD command sequences, but its program representation remains linear and lacks explicit structural reasoning. CADmium [15] formulates CAD generation as code synthesis, producing executable CAD programs through LLMs. However, similar to other program-based approaches, it relies on sequential command representations that do not explicitly capture hierarchical geometric structure. Query2CAD [40] generates executable FreeCAD Python macros using an LLM with iterative refinement loops, focusing on script-based CAD generation rather than direct geometric modeling. Despite their differences in modality and modeling paradigms, these approaches share several inherent limitations. Point clouds are sparse and lack structural information, making it difficult to extract reliable geometric guidance. Meshes are non-parametric and struggle to encode semantics or hierarchy. Multi-view images are vulnerable to occlusions and weak supervision. CAD operation sequences, while informative, are typically linear and fail to explicitly represent geometric composition or structural constraints. As a result, these representations fall short in modeling the constructive logic of CAD, limiting the precision, editability, and structural consistency of the generated outputs.

Different representations for CAD generation. Boundary Representation (B-Rep) defines geometry and topology by explicitly specifying the vertices, edges, and faces forming a shape's boundary. Many methods generate CAD models using B-Rep, such as BrepGen [41], SolidGen [42], and so on [43–46]. However, B-Rep lacks structural semantics, which hampers its use in data-driven modeling. It is difficult to extract construction logic, interpret topology, or apply semantic control based on B-Rep. What is more, GeoCAD [36] learns to generate parametric CAD programs by aligning geometric priors with textual descriptions. GeneratingCAD [37] uses vision-language models to generate executable CAD code, whereas Img2CAD [47] predicts a discrete structural

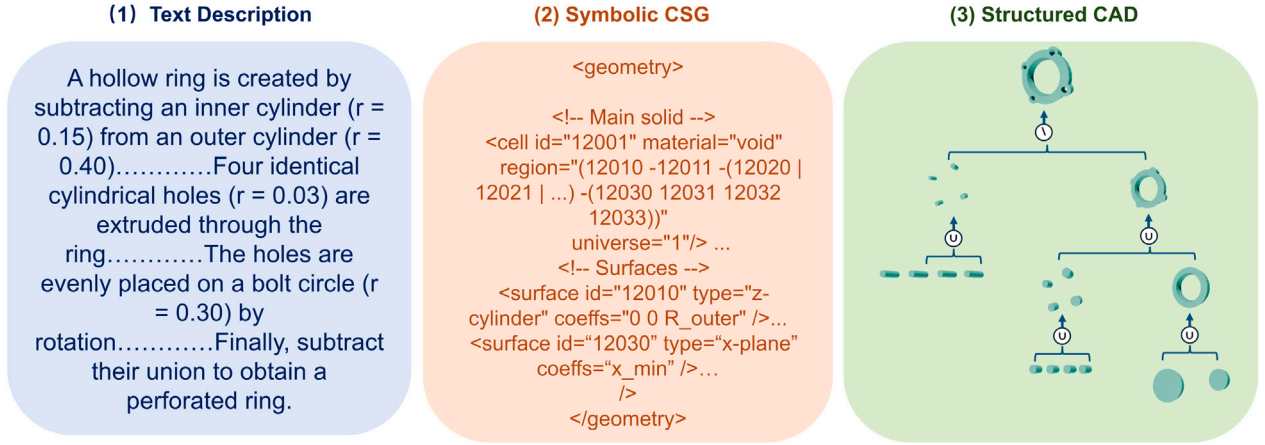


Fig. 2. Overview. The natural-language description (left) provides an overview of the object. The corresponding CSG script (middle) expresses the model using analytic primitives and Boolean operations. The constructive hierarchy (right) shows how the final shape is built from these components. Only key primitives and simplified text are shown for clarity.

layout before regressing continuous parameters. Sketch-and-extrusion frameworks [48–54] construct geometry through procedural operation sequences, mirroring standard engineering workflows. DeepCAD [13] exemplifies this approach by modeling 3D shapes as 2D profiles subject to extrusion or revolution. Building on this foundation, Text2CAD [14] extends the paradigm to natural language, employing a Transformer to synthesize CAD command sequences directly from text. While this paradigm aligns well with real-world design practices, it introduces challenges for learning-based modeling, as sketch-extrusion programs are represented as order-sensitive sequences with implicit dependencies, making it difficult to explicitly capture hierarchical geometric structure. Moreover, although modern CAD systems maintain richer dependency structures, existing learning-based approaches often rely on simplified sequential representations for modeling, without explicitly encoding hierarchical dependencies.

CAD datasets. Several CAD datasets have been introduced recently [40, 55–58], but many of them fail to learn explicit construction logic. The ABC dataset [59], for example, contains high-quality B-Rep and STEP models that are useful for geometric processing, yet offer no information about design procedures or hierarchical structure. The Fusion 360 Gallery dataset [60] captures real modeling workflows, but the logs are noisy and inconsistent across users. Datasets of DeepCAD [13] and Text2CAD [14] provide sketch-and-extrusion sequences, but these sequences remain low-level and vary widely in style, limiting their ability to serve as a unified and expressive representation of design logic. Although CSG offers a clear and interpretable format, large-scale collections of CSG programs with semantic annotations are still limited.

3. Method

3.1. Overview

Text2CSG targets the generation of interpretable and structurally coherent CAD models from natural language descriptions. We formulate this as a two-stage framework that maps text into executable CSG programs, explicitly preserving the geometric construction logic of the target model (see Fig. 3).

In the first stage, a structure-aware CSG decoder is trained to produce syntactically valid token sequences that encode the hierarchical construction tree of a model. This training uses only CSG data, and the decoder learns structural regularities through a tree-guided attention mechanism based on parent–child and sibling relationships, together with a structural loss that reinforces hierarchical consistency.

In the second stage, a text-to-CSG alignment module maps natural-language inputs into the latent space of the pretrained decoder. Semantic embeddings obtained from a frozen language model (BERT) are projected into this space and incorporated into decoding via layer-wise cross-attention, where CSG tokens query the textual representation. By separating geometric program learning from language grounding, Text2CSG maintains the structural control offered by symbolic program generation while leveraging the semantic generality of pretrained language models. The resulting system supports efficient training and yields syntactically correct, structurally reliable CAD programs from open-ended natural-language prompts.

In our method, CAD models are represented using CSG. Each model is expressed as a Boolean composition of analytic primitives, forming a hierarchical construction tree (Fig. 4). For implementation, we adopt the Monte Carlo N-Particle (MCNP) style CSG representation [61], in which region definitions explicitly encode Boolean operations and signed primitives. This representation offers a clean symbolic structure that can be deterministically serialized into tokens and re-constructed, making it well suited for program-level supervision and sequence generation.

3.2. Stage 1: Structure-aware CSG program generation

3.2.1. Structured token representation of CSG trees

To enable efficient learning from structured CSG data, we design a compact and hierarchical encoding scheme that transforms a CSG construction tree into a discrete token sequence suitable for Transformer-based models. Each node in the CSG tree represents a primitive shape or a Boolean operator, which is encoded as a fixed-length vector containing both semantic and structural information. In this setting, each signed half-space is defined by an analytic surface such as a plane, cylinder, or quadric. These half-spaces form the leaf nodes of the constructive hierarchy, while combinations of half-spaces (forming primitives) and Boolean operators (unions and intersections) serve as internal nodes. The resulting representation induces a shallow but wide n -ary Boolean tree, which provides a structured foundation for tokenization and hierarchical modeling. To preserve the hierarchical structure of the CSG tree, we serialize it in preorder and represent each node as a fixed-length token vector:

$$\mathcal{T} = \{t_1, t_2, \dots, t_N\}, \quad (1)$$

where each token t_i is defined as:

$$t_i = [T_i, S_i, SId_i, PId_i, D_i, Id_i], \quad (2)$$

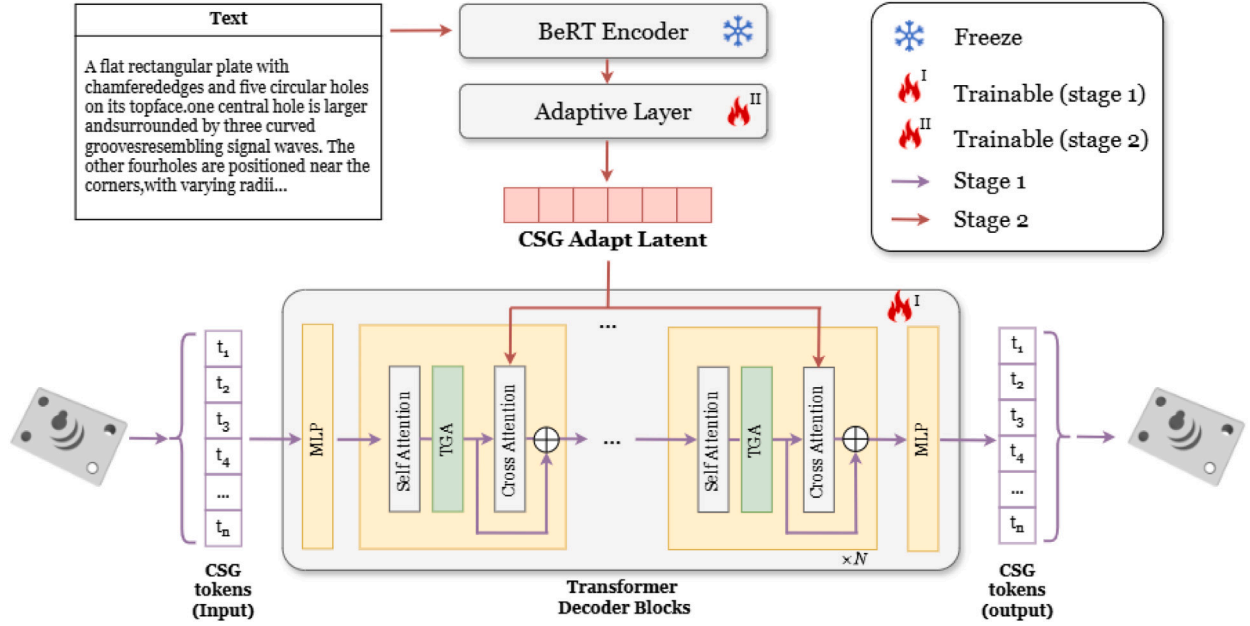


Fig. 3. Pipeline of the Text2CSG framework. Training proceeds in two stages: (1) a CSG decoder with hierarchical and syntactic supervision is trained in stage 1, and (2) aligning textual semantics with CSG representations. A frozen language encoder extracts semantic features, which are projected into the CSG latent space via a cross-attention module to condition the synthesis of executable CSG programs.

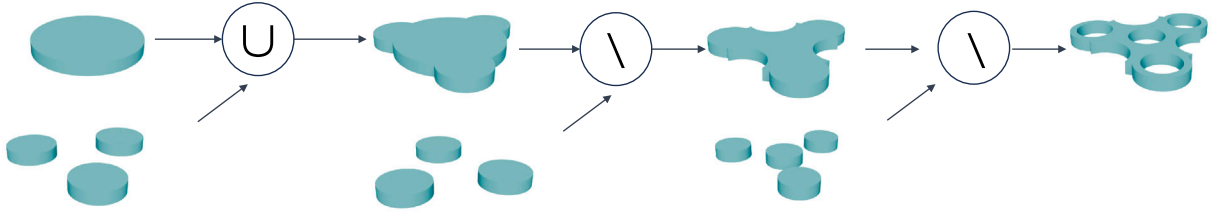


Fig. 4. CSG builds CAD models by recursively applying Boolean operations to primitives, employing operators such as union ($\cup$) and difference ($\setminus$). This representation yields a hierarchical construction tree that preserves both geometric data and symbolic design logic, ensuring the resulting models are interpretable and amenable to parametric editing.

where T_i indicates the node type, S_i encodes the sign of the half-space, Std_i refers to the surface index (for leaf nodes), PId_i specifies the parent node ID, D_i represents the node's depth in the tree, and Id_i is the unique identifier of the node. To delineate logical segments within the token sequence, we wrap each complete CSG model, comprising one or more cells with two special tokens: SOL (Start of Logic) and EOL (End of Logic). This ensures that all Boolean expressions, including those involving multiple primitives, are processed as a unified structure. Such boundary markers preserve semantic integrity and maintain compatibility with Transformer-based sequence models, enabling the model to recognize the start and end of each independent CSG program. Geometric parameters associated with nodes are normalized to the range $[-1, 1]$ and then uniformly quantized into 8-bit integers:

$$\hat{x} = \left\lfloor \frac{(x+1)}{2} \cdot 255 \right\rfloor, \quad x \in [-1, 1]. \quad (3)$$

This quantization procedure provides a compact yet sufficiently accurate discrete encoding of continuous geometry, supporting efficient storage and reconstruction. Compared to mesh-based or point-based representations that rely on thousands of vertices or samples, our symbolic token representation is significantly more compact and semantically structured. Moreover, this discrete tokenization is inherently compatible with Transformer-based generative models and can be seamlessly integrated into language-conditioned generation pipelines. By representing all symbolic operations and numerical parameters as discrete tokens, the CSG sequence can be directly aligned with textual

embeddings, enabling joint training and decoding for tasks such as text-to-CAD generation.

3.2.2. Tree-guided attention and structural loss

To better capture the hierarchical structure inherent in CSG trees, we incorporate a tree-guided attention mechanism into the Transformer decoder, inspired by previous work on tree-structured attention and attention-based tree traversal in NLP [62,63]. Unlike standard self-attention, which treats all tokens equally, our approach injects structural priors by explicitly modeling the *parent-child* and *sibling* relationships within the CSG tree. Each token t_i in the CSG sequence is augmented with two directed attention pathways: *parent attention*, which incorporates the parent node $p(i)$ for top-down propagation of structural information; and *sibling attention*, which connects node i with its preceding siblings $s(i)$ under the same parent to capture lateral geometric relations. These pathways are implemented through binary masks $\mathbf{M}^{\text{parent}}$ and $\mathbf{M}^{\text{sibling}}$ that restrict attention heads:

$$\mathbf{A}^{\text{guided}} = \text{Softmax} \left(\frac{(\mathbf{Q}\mathbf{W}_q)(\mathbf{K}\mathbf{W}_k)^\top}{\sqrt{d}} \odot \mathbf{M} \right), \quad (4)$$

where $\mathbf{M}$ combines $\mathbf{M}^{\text{parent}}$ and $\mathbf{M}^{\text{sibling}}$, and $\odot$ denotes element-wise masking. The guided attention output is fused with the standard self-attention output via residual addition:

$$\mathbf{H} = \text{LayerNorm}(\mathbf{H}^{\text{self}} + \mathbf{H}^{\text{guided}}), \quad (5)$$

where $\mathbf{H}^{\text{self}}$ is from vanilla self-attention and $\mathbf{H}^{\text{guided}}$ incorporates structural guidance.

To further regularize generation and improve structural fidelity, we incorporate a structural loss that penalizes inconsistencies in the predicted CSG tree, including incorrect parent-child assignments, sibling orderings, and syntax violations. The total structural loss is defined as:

$$\mathcal{L}_{\text{struct}} = \lambda_1 \cdot \mathcal{L}_{\text{parent}} + \lambda_2 \cdot \mathcal{L}_{\text{sibling}} + \lambda_3 \cdot \mathcal{L}_{\text{syntax}}, \quad (6)$$

where each term supervises a specific aspect of the CSG structure. Although effective, applying this loss from the beginning of training may interfere with early learning, as the decoder might not yet produce a meaningful syntax or structure. To mitigate this, we adopt a curriculum-style training strategy. Specifically, we first pretrain the CSG decoder using only standard cross-entropy loss $\mathcal{L}_{\text{CE}}$ to ensure basic token correctness. After a warm-up period of T_w steps, we gradually introduce the structural loss using a linear ramp-up schedule:

$$\mathcal{L}_{\text{pretrain}} = \mathcal{L}_{\text{CE}} + \alpha(t) \cdot \mathcal{L}_{\text{struct}}, \quad \alpha(t) = \max\left(0, \min\left(1, \frac{t - T_w}{T_s}\right)\right), \quad (7)$$

where t is the current training step, and T_s is the ramp-up duration. This progressive schedule allows the model to first learn valid syntax before enforcing hierarchical consistency, ultimately improving both generation quality and structural alignment.

3.2.3. Token-to-CSG: Invertible reconstruction of CAD geometry

To ensure that our token representation remains both semantically equivalent and fully executable, we implement an invertible reconstruction pipeline capable of transforming a token sequence back into a valid MCNP-style CSG expression in XML format. Given a sequence delimited by the special tokens SOL and EOL, we first reconstruct the underlying syntax tree by linking each node to its parent through the stored Pid_i references. The recovered tree is then recursively traversed to rebuild the Boolean expression, preserving operator precedence and restoring the correct sign annotations for all signed half-spaces. The surface identifiers in the token sequence are subsequently mapped back to their corresponding analytic surface definitions, allowing full geometric attributes — such as element type, coefficients, and boundary constraints — to be restored. Finally, the reconstructed cells and surface records are assembled into a syntactically valid XML document, yielding a complete and executable CSG program identical to the original. The reconstructed output retains full semantic equivalence with the original model, including both the logical structure and geometric parameters. This invertibility property enables effective training, generation, and editing of CAD models using a token-based representation.

3.3. Stage 2: Language-guided CSG generation via cross-modal alignment

In the second stage of our framework, we aim to bridge the structural priors learned from pure CSG data with high-level semantic inputs, enabling the generation of syntactically valid and semantically grounded construction programs. Unlike prior approaches that treat language as the primary input modality throughout training, we adopt a structurally guided training strategy: the CSG decoder, pretrained in Stage 1 to model program syntax and hierarchy, is now conditioned on textual information via cross-modal attention.

Semantic embedding alignment. Given a natural language description x^{text} , we first extract high-dimensional semantic features using LLM:

$$\mathbf{E}^{\text{text}} = \text{LLM}(x^{\text{text}}) \in \mathbb{R}^{L \times d}, \quad (8)$$

where L is the number of text tokens and d is the hidden size. We then use a learnable linear projection to align the language features to the decoder's latent space:

$$\tilde{\mathbf{E}}^{\text{text}} = \mathbf{E}^{\text{text}} \cdot \mathbf{W}_{\text{proj}}, \quad \mathbf{W}_{\text{proj}} \in \mathbb{R}^{d \times d}. \quad (9)$$

Cross-attention conditioning. We adopt a Transformer decoder architecture augmented with layer-wise cross-attention. At decoding layer l , the CSG token representation $\mathbf{Z}^{(l)}$ attends to the aligned language embedding via:

$$\mathbf{A}^{(l)} = \text{Softmax}\left(\frac{\mathbf{Q}^{(l)}(\tilde{\mathbf{E}}^{\text{text}})^{\top}}{\sqrt{d}}\right)\tilde{\mathbf{E}}^{\text{text}}, \quad (10)$$

where $\mathbf{Q}^{(l)} = \mathbf{Z}^{(l)} \cdot \mathbf{W}_Q$ is the query matrix, and $\tilde{\mathbf{E}}^{\text{text}}$ serves as both key and value. The cross-attention output is combined with the self-attention result and passed to the feed-forward network:

$$\mathbf{Z}^{(l+1)} = \text{FFN}(\text{LayerNorm}(\mathbf{Z}^{(l)} + \mathbf{A}^{(l)})). \quad (11)$$

Language-conditioned decoding. During training, both the CSG tokens and text embeddings are used as input. At inference time, only the natural language prompt is provided, and the model autoregressively generates the full CSG sequence. Thanks to the structural competence learned in Stage 1 and the semantic alignment introduced in this stage, the decoder can generate syntactically valid and semantically coherent programs. This two-stage framework decouples geometry modeling from semantic grounding, enabling robust and interpretable text-to-CAD generation.

4. Experiments

4.1. CSG dataset construction

Existing CAD datasets often lack structured supervision and hierarchical representations. We summarize the comparison of major CAD datasets in Table 1. For instance, the widely used ABC dataset [59] provides high-precision models in B-Rep format, which are well-suited for low-level geometry tasks such as surface fitting or feature extraction. However, these representations do not capture the underlying constructive logic, limiting their utility for interpretable or generative modeling. The DeepCAD [13] dataset partially addresses this by introducing sketch- and extrusion-based modeling sequences, while Text2CAD [14] further augments them with natural language annotations. Nonetheless, there is still no large-scale dataset that represents CAD models in a unified, symbolic, and semantically aligned CSG format. To address this gap, we create a structured CSG dataset by processing and reorganizing existing CAD resources, specifically ABC [59] and Text2CAD [14]. The geometric models are sourced from ABC, while natural-language descriptions are adapted from Text2CAD. Through systematic filtering, validation, and conversion, we transform these inputs into a unified CSG-based representation with explicit Boolean construction hierarchies.

We adopt a validation pipeline centered on CSG decomposition, focusing on the validity and structural consistency of the resulting symbolic CSG representations. During the conversion process using the Geouned framework [64], a subset of models cannot be reliably decomposed into consistent Boolean structures due to ambiguities in primitive extraction or instability in geometric reconstruction. These cases are filtered through a combination of conversion failure detection, targeted manual inspection, and stochastic volume validation, as detailed in Table 2. Such issues mainly arise from the inherent difficulty of reconstructing consistent CSG decompositions from boundary representations, rather than from intrinsic defects in the original dataset. We use FreeCAD to perform Boolean operations and manifold checks, and convert only validated files into symbolic CSG expressions using the Geouned framework [64], which follows the MCNP-style representation where solids are described as Boolean combinations of analytic half-spaces (planes, cylinders, and quadrics). Boolean combinations of analytic half-spaces can reproduce various solid components, including polyhedral shapes defined by planes, cylinder elements, and spherical regions modeled through quadratic surfaces. MCNP-style CSG represents solids as Boolean expressions over analytic half-spaces. These

Table 1

Comparison of major CAD datasets highlighting representation type, hierarchical structure, and availability of text annotations.

Dataset	Representation	Hierarchy	Text annotation
ABC	B-Rep solids	No	No
DeepCAD [13]	Sketch + Extrusion programs	Limited (linear)	No
Text2CAD [14]	Sketch + Extrusion programs	Limited (linear)	Yes
Query2CAD [40]	Parametric primitives + constraints	Limited (constraint graph)	Yes
SketchGraphs [56]	2D constraint graphs	Yes (graph structure)	No
Open-ECD [55]	Engineering B-Rep (STEP)	No	No
Text2CSG (ours)	Symbolic CSG programs	Full Boolean tree hierarchy	Yes

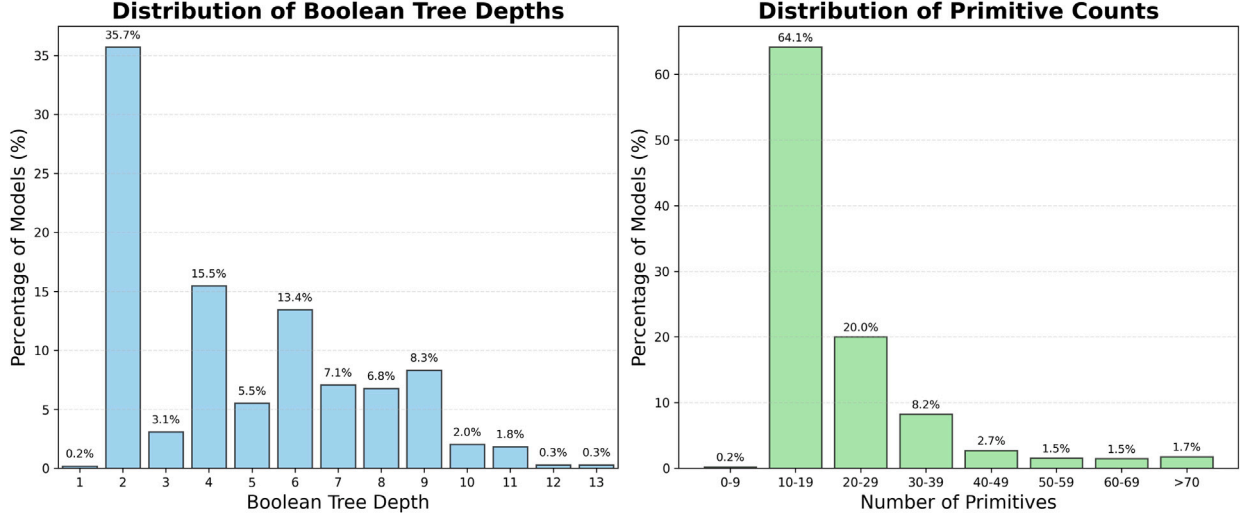


Fig. 5. Dataset statistics. Left: distribution of Boolean tree depths, reflecting the hierarchical complexity of constructive expressions. Right: distribution of primitive counts per model.

Table 2

Breakdown of the dataset filtering process during CSG dataset construction.

Stage	Discarded models	Ratio (%)
GEOUNED conversion failure	2173	6.94
Manual inspection (FreeCAD)	768	2.45
Stochastic volume validation	2843	9.09
Total discarded	5784	18.48
Final dataset size	25,500	81.52

expressions can be parsed into a constructive hierarchy comprising unions, intersections, and grouped primitive constraints. This hierarchy enables per-model structural tokenization and hierarchical supervision, allowing models to jointly reason about geometry and language. Compared to prior datasets, our representation is interpretable and directly compatible with symbolic program generation, which supports end-to-end learning of editable and semantically meaningful CAD programs.

The resulting dataset contains 25,500 validated CSG models. Fig. 5 summarizes its structural statistics: the Boolean expression trees exhibit moderate depth, with an average depth of 4.77 and most models ranging between depths 2 and 9. The average number of primitives per model is 21.95, with a long-tailed distribution reflecting increasing geometric complexity.

4.2. Implementation details

Our implementation is based on PyTorch and trained on a NVIDIA A100 GPU. For Stage 1, we use a 6-layer Transformer decoder with 8 attention heads and a hidden size of 512. The token vocabulary includes

geometry types, Boolean operations, and quantized parameters, totaling approximately 400 unique tokens. We pretrain the CSG decoder for 200k steps. During the initial $T_w = 20k$ steps, we use only cross-entropy loss. After this warm-up phase, we introduce the structural supervision loss $\mathcal{L}_{\text{struct}}$ using a linear ramp-up over $T_s = 10k$ steps (see Eq. (7)). We set the loss weights as $\lambda_1 = \lambda_2 = 0.4$, and $\lambda_3 = 0.2$. In Stage 2, we freeze the decoder and train the text-to-CSG alignment module. We use a pretrained BERT-base [65] model as the frozen LLM encoder and apply a learnable linear projection to map its output into the decoding space. Each Transformer layer includes layer-wise cross-attention between CSG tokens and text embeddings. The training process takes less than 20 h.

4.3. Evaluation

We evaluate our model along two main objectives: (1) semantic alignment between natural language inputs and the generated CSG programs, and (2) geometric quality and structural validity of the reconstructed shapes, particularly for complex, multi-component designs. These objectives reflect our goal of producing interpretable, semantically consistent, and structurally sound CSG representations from text. We conduct both quantitative evaluations and qualitative comparisons. Geometric accuracy is measured using Chamfer Distance (CD) and F1-score, which assess the consistency between reconstructed shapes and ground-truth geometry. To evaluate structural validity, we report Invalidity Ratio (IR), Self-Intersection Ratio (SIR), and Watertightness. IR measures the fraction of generated CAD programs that fail to produce valid geometry, SIR quantifies the extent of self-intersecting surfaces, and Watertightness indicates whether the generated shapes form closed, manifold solids.

Table 3

Quantitative comparison of CAD generation methods on our dataset. Chamfer Distance (CD) is scaled by $\times 10^3$. Our method achieves competitive geometric accuracy while consistently outperforming baselines in structural validity metrics. We additionally include DeepCAD and CADmium, along with Watertightness and SIR metrics.

Metric	DeepCAD	Text2CAD	CADmium	Ours
Median CD ↓	29.77	0.44	0.43	0.36
Mean CD ↓	94.82	25.49	25.98	24.13
F1-score (%) ↑	80.13	79.27	81.01	86.30
Invalidity Ratio (IR, %) ↓	9.73	1.62	2.35	0.42
Watertightness (%) ↑	86.37	90.52	88.43	91.23
Self-Intersection Ratio (SIR, %) ↓	0.09	0.05	0.14	0.03

Table 4

Distribution of CD ($\times 10^3$) between generated models from our method and GT on the evaluation set.

CD range	[0, 0.2]	(0.2, 0.4]	(0.4, 1.0]	>1.0
Ratio (%)	17.24	36.54	33.52	12.70

Qualitative comparison. Fig. 6 presents a visual comparison between our method and representative baselines, including Text2CAD and CADmium, on text-described CAD models with complex, multi-component structures. While both methods can produce reasonable results in simpler cases, they often struggle with scenarios involving intersecting components or multi-part structures that require consistent geometric organization. In particular, their underlying representations rely on sequential or implicit modeling of geometry, which makes it difficult to consistently capture hierarchical relationships among components. In contrast, our method explicitly models the constructive process through a tree-structured CSG representation. This enables more accurate handling of compositional semantics and enforces structural validity during generation, leading to more reliable and faithful reconstructions, especially in multi-component cases.

Quantitative evaluation. Table 3 reports the quantitative comparison between Overall, our approach achieves strong performance across both geometric accuracy and structural validity metrics. In terms of geometric accuracy, our method obtains lower Chamfer Distance and a higher F1-score, indicating more accurate alignment with the ground-truth geometry. The gap between the mean and median CD for all methods is caused by a small number of severe failure cases. As noted in Text2CAD [14], these failures are largely attributed to noise in the automatically generated textual annotations. Specifically, LLaVA-NeXT [66] is used to generate descriptions, but its sensitivity to viewpoint variations and the dataset's strong shape imbalance can lead to inaccurate annotations for complex models. These noisy descriptions introduce unstable supervision signals, producing heavy-tailed errors that significantly increase the mean CD while having a limited effect on the median. Table 4 further characterizes this behavior for our method. More than half of the predictions lie below CD 0.4, and 83% fall below 1.0, while only 12.70% belong to the heavy-tailed region ($CD > 1$), indicating that the elevated mean CD is driven by a small fraction of difficult cases rather than overall reconstruction quality. In addition to geometric accuracy, our method demonstrates clear advantages in structural validity, achieving the lowest Invalidity Ratio (0.42%), the highest Watertightness (91.23%), and the lowest SIR (0.03%) among all compared methods. Although CADmium benefits from improved textual annotations and a more flexible text-driven formulation that does not require explicit alignment with structured CAD sequences such as CSG or sketch-based programs, its performance on structural metrics does not consistently surpass that of Text2CAD, and remains inferior to our method, suggesting that improving language understanding alone is insufficient for reliable CAD generation. A key limitation lies in the lack of explicit structural modeling. DeepCAD, Text2CAD, and CADmium

primarily rely on sequence-based representations, where geometric and topological constraints are only implicitly learned. This results in limited inductive bias toward valid and well-structured CAD programs, making them more prone to invalid or inconsistent outputs. In contrast, Text2CAD achieves a relatively low Invalidity Ratio due to its carefully designed vector representation. However, its linear formulation still lacks explicit hierarchical structure, limiting its ability to model multi-component geometries. Our approach addresses these limitations by explicitly modeling CAD construction as a tree-structured CSG process, where both geometry and topology are directly encoded, providing stronger constraints during generation and leading to more structurally consistent and reliable CAD outputs. Overall, these results suggest that our method provides a better balance between geometric fidelity and structural correctness, particularly in multi-component scenarios.

4.4. Qualitative comparison with recent CAD generation methods

Recent work on data-driven CAD modeling spans a wide range of symbolic representations and generation paradigms. To contextualize the design choices behind Text2CSG, we compare our framework with representative categories of existing approaches, including unconditional program generators (DeepCAD [13]), image-conditioned CAD code generators (CADCoder [67]), text-conditioned CAD program synthesis models (Text2CAD [14]), LLM-based CAD program generation methods that directly map text to CAD sequences (CADmium [15]), hierarchy-aware representation learners based on fine-tuned LLMs (FlexCAD [20]), multimodal LLM-based CAD understanding systems (CAD-MLLM [19]), and CSG-style code generators such as Do not Mesh With Me (DMwM) [68]. Several recent methods also focus on multimodal inputs or task-specific generation strategies. For instance, GeoCAD [36] generates parametric CAD programs by aligning geometric priors with textual descriptions, CADGPT [39] directly parses text into executable CAD commands, GeneratingCAD [37] uses vision-language models to generate executable CAD code, Text-to-CAD [35] incorporates a visual-feedback loop in LLMs to iteratively refine CAD command sequences, Img2CAD [47] predicts discrete structural layouts from images and regresses continuous parameters, and DrawStepByStep [51] reconstructs CAD construction sequences from point clouds using diffusion-based modeling. The comparison follows the structure of input, design, and output used in Table 5 and examines five aspects that characterize practical symbolic CAD generation: (a) Representation and (b) input modality, which determine the type of geometric source a method operates on and whether it supports natural-language design intent; (c) Model architecture, which reflects the underlying modeling paradigm (e.g., Transformer-based, LLM-based, or geometry-aware methods); (d) Fine-tuning, indicating whether a method needs task-specific adaptation on top of a pretrained backbone; and (e) Output, which reflects the form and interpretability of the final symbolic output. In addition, different approaches vary in their ability to ensure structural validity, which is not always explicitly enforced in sequence-based or LLM-based formulations.

As summarized in Table 5, Text2CSG uniquely integrates natural-language controllability, explicit construction-logic modeling, and inference without task-specific fine-tuning. Unlike prior text-to-CAD systems that flatten programs into linear token streams, multimodal LLM-based approaches that overlook structural validity, or CSG-style code generators such as DMwM that encode constructive semantics only implicitly in linear code and require substantial fine-tuning, Text2CSG directly predicts a tree-structured CSG program with well-defined Boolean dependencies. This design yields interpretable and structurally valid symbolic outputs, specifically *CSG Boolean trees*, while avoiding the computational and practical overhead of task-specific fine-tuning.

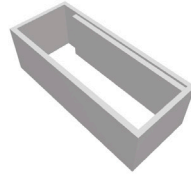
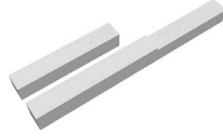
**Text
Description
(Simplified)**

Construct a **3D cross** by intersecting **two rectangular bars** perpendicularly. The horizontal bar ($0.643 \times 0.107 \times 0.107$) serves as the base, while the vertical bar ($0.627 \times 0.107 \times 0.107$) features a **notch for interlocking**. Result: T-shaped structure with the top bar extending beyond the bottom.

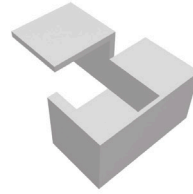
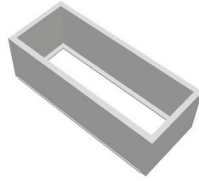
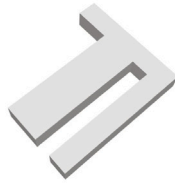
Create a **rectangular container** with outer dimensions $0.75 \times 0.3088 \times 0.2404$ units. Add an **inner rectangular cavity** with dimensions $0.6176 \times 0.2647 \times 0.011$ units deep, forming a **hollow box structure**.

Construct a mechanical mounting base by combining two extruded profiles with **rounded corners**. The primary body is a rectangular plate ($0.75 \times 0.41 \times 0.31$) featuring **four corner mounting holes** and a larger central aperture. A secondary square boss ($0.43 \times 0.43 \times 0.38$) with a **single central hole** is added atop the base.

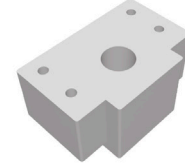
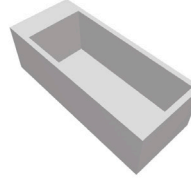
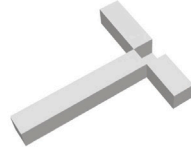
Text2CAD



CADmium



Ours



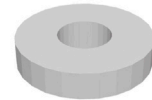
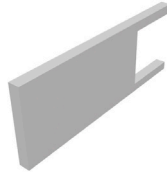
**Text
Description
(Simplified)**

Construct a T-shaped pipe junction by intersecting **three hollow cylinders**. The vertical trunk is formed by two aligned cylindrical segments, while a third horizontal cylinder **intersects** them perpendicularly. Each segment ($0.36 \times 0.36 \times 0.57$) features a concentric circular profile to define the pipe wall thickness.

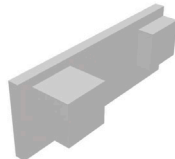
Construct a composite assembly consisting of a **thin rectangular base plate** ($0.75 \times 0.28 \times 0.04$) supporting **two distinct rectangular blocks**. A larger, taller block ($0.22 \times 0.21 \times 0.14$) is extruded on the right side of the base, while a smaller, shorter block ($0.17 \times 0.19 \times 0.06$) is positioned on the left side.

Construct a stepped cylindrical component by stacking **two concentric hollow discs**. The larger base ring ($0.59 \times 0.59 \times 0.12$) supports a smaller, **thinner upper ring** ($0.43 \times 0.43 \times 0.04$). Both parts share the **same inner diameter**, creating a continuous central bore through the assembly.

Text2CAD



CADmium



Ours



Fig. 6. Qualitative comparison with Text2CAD [14] and CADmium [15]. Our approach more reliably produces structurally valid CAD outputs, while Text2CAD and CADmium more frequently fail on descriptions involving multiple components. For readability, the input text descriptions shown here are simplified; the actual inputs contain more detailed geometric specifications.

Table 5

Qualitative comparison of representative CAD generation frameworks across representation, input modality, model architecture, fine-tuning requirement, and output format. Our method differs from prior approaches by modeling CAD generation as a hierarchical CSG tree with explicit constructive logic, while avoiding the need for fine-tuning.

Method	Representation	Input	Architecture	Fine-tuning	Output
DeepCAD [13]	Sketch + Extrusion commands	Sketch + Extrusion sequence	Transformer-based	No	Sketch + Extrusion program
CAD-Coder [67]	CadQuery	Image	LLM-based	Yes	CadQuery Script
Text2CAD [14]	Sketch + Extrusion commands	Text	Transformer-based	No	Sketch + Extrusion program
Img2CAD [47]	Discrete structure + Parameters	Images	Geometry-aware network	Yes	Parametric CAD program
Text-to-CAD [35]	CAD command sequence	Text + Rendering feedback	LLM-based	Yes	Refined CAD command sequence
FlexCAD [20]	Structured CAD text	CAD model + Text	LLM-based	Yes	Structured CAD text
GeoCAD [36]	Primitive-based CAD program	Text + Geometry	Geometry-aware network	Yes	Parametric CAD program
CAD-MLLM [19]	CAD command sequence	Text/Image/Points	Multimodal LLM	Yes	CAD command sequence
Do not mesh with me [68]	CSG code	Text + CSG context	LLM-based	Yes	CSG-based Python script
CADGPT [39]	CAD command sequence	Text	LLM-based	Yes	Executable CAD commands
DrawStepByStep [51]	CAD construction sequence	Point cloud	Transformer-based	Yes	CAD operation sequence
GeneratingCAD [37]	CAD code	Image/Text	LLM-based	Yes	Executable CAD code
CADmium [15]	Sketch + Extrusion commands	Text	LLM-based	Yes	Sketch + Extrusion program
Text2CSG (ours)	CSG (Tree)	Text	Structured transformer	No	CSG Boolean tree

Table 6

Ablation on structure modeling components on our dataset.

Method	IR (%) ↓	Watertightness (%) ↑	SIR (%) ↓
Ours w/o Tree-Guided attention & w/o Structural loss	1.82	89.47	0.07
Ours w/o Tree-Guided attention	1.44	89.92	0.06
Ours w/o Structural loss	0.87	90.43	0.04
Ours (full)	0.42	91.23	0.03

Table 7

Ablation on loss weighting strategies.

Loss weighting strategy	IR (%) ↓	Watertightness (%) ↑	SIR (%) ↓
Static	1.36	90.03	0.06
Two-stage	0.91	90.67	0.04
Progressive (ours)	0.42	91.23	0.03

4.5. Ablation studies

Ablation on structure-aware modeling. To assess the contribution of our structure-aware components, we perform ablation on the Tree-Guided Attention (TGA) and the structural loss $\mathcal{L}_{\text{struct}}$. As shown in Table 6, removing either module increases the IR, confirming that both the parent/sibling attention and the topological supervision are essential for syntactic validity. In addition, both components improve Watertightness and reduce the SIR, indicating better geometric consistency. Notably, removing the structural loss leads to a more significant degradation across all metrics, highlighting its critical role in enforcing topological correctness, while Tree-Guided Attention further enhances structural coherence by modeling hierarchical dependencies. These results suggest that the observed improvements stem primarily from the proposed structure-aware modeling, as the explicit modeling of hierarchical relationships and topological constraints plays a central role in achieving reliable outputs.

Ablation on progressive loss weighting. To verify the effectiveness of the progressive loss scheduling introduced in Section 3.2.2, we compare three loss weighting strategies: (1) static, where all loss terms remain fixed; (2) two-stage, which first optimizes token-level objectives and later activates structural and geometric terms, and (3) progressive (ours), which gradually increases the influence of structural and geometric supervision over training (see Table 7).

4.6. Interpretability and editing analysis

We further analyze the interpretability of the generated CSG programs and their capability for structured editing. As shown in Fig. 7, our model predicts hierarchical CSG trees where geometric primitives

are composed through explicit Boolean operations. This representation establishes a direct correspondence between tree nodes and geometric components, making the construction process transparent and interpretable. Moreover, the explicit hierarchical structure enables intuitive and localized editing. As illustrated in Fig. 8, modifying parameters or subtrees leads to predictable geometric changes, including parameter adjustment, component removal, and component addition. Such edits remain spatially localized and do not affect unrelated parts of the model. In contrast, sequence-based representations often entangle geometry and construction order, making local editing less stable and harder to control. These results demonstrate that our approach not only improves generation quality but also provides a more interpretable and controllable representation for downstream CAD editing. Furthermore, the explicit structure of the CSG representation also provides a natural interface for potential training-free editing strategies, for example, by modifying specific primitives or subtrees through high-level instructions, potentially guided by external language models, without requiring additional retraining.

4.7. Discussion on failure cases

Although the method performs well in most cases, some failure cases remain. As discussed in Section 4.3, some errors are due to limitations in the automatically generated text annotations. In particular, when conditioned on multi-view image inputs, LLaVA-NeXT [66] is sensitive to perspective distortions across views, which may introduce inaccuracies into the annotations. Moreover, there is currently no reliable mechanism to assess the quality of these generated descriptions, further affecting the robustness of the training data. We also observe that failures may occur when the descriptions are too generic or not precise enough, causing the model to produce incomplete or incorrect geometric structures. Fig. 9 shows that different textual descriptions targeting the same shape can lead to very different generation outcomes. In the case of the three-step stair shown in Fig. 9(a) and (b), an imprecise description fails to recover the intended stepped structure, while a more detailed description that emphasizes geometric structure allows the model to generate the correct result. The same issue arises in the table examples in Fig. 9(c) and (d). The automatically generated descriptions seldom use object-level terms such as “table”. Instead, they tend to describe shapes using basic geometric primitives, such as rectangular prisms or cylinders, together with low-level construction operations such as sketching, extrusion, rotation, and translation. Although this description style is often adequate for simple solids, it frequently omits essential geometric parameters, including dimensions, relative offsets, and part proportions. Such omissions make it difficult for the model to correctly instantiate multi-part objects with functional structure. Consequently, objects like tables are more likely to be generated incorrectly or incompletely when the descriptions lack explicit parameterization,

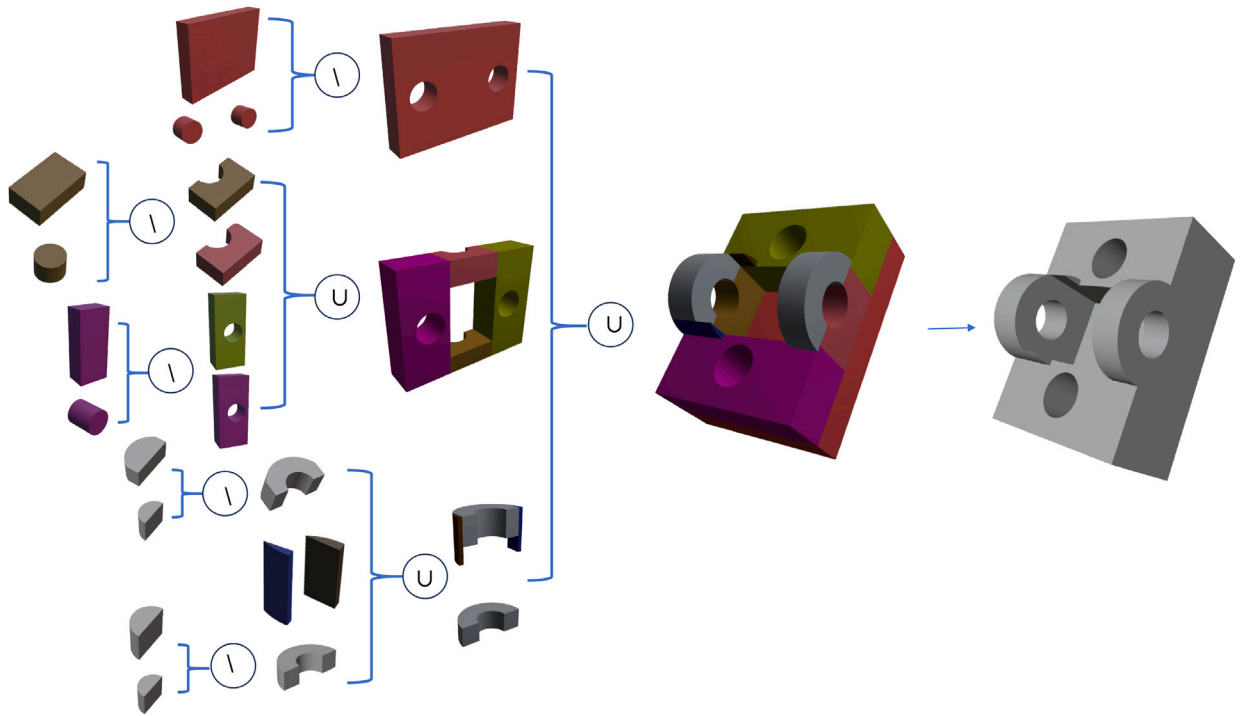


Fig. 7. Visualization of the hierarchical structure of the learned CSG representation and its support for localized CAD editing. From left to right: (a) the hierarchical CSG tree, where primitives are recursively composed through Boolean operations; (b) the corresponding colored geometric decomposition, in which semantically meaningful components align with substructures of the tree; and (c) the reconstructed geometry. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

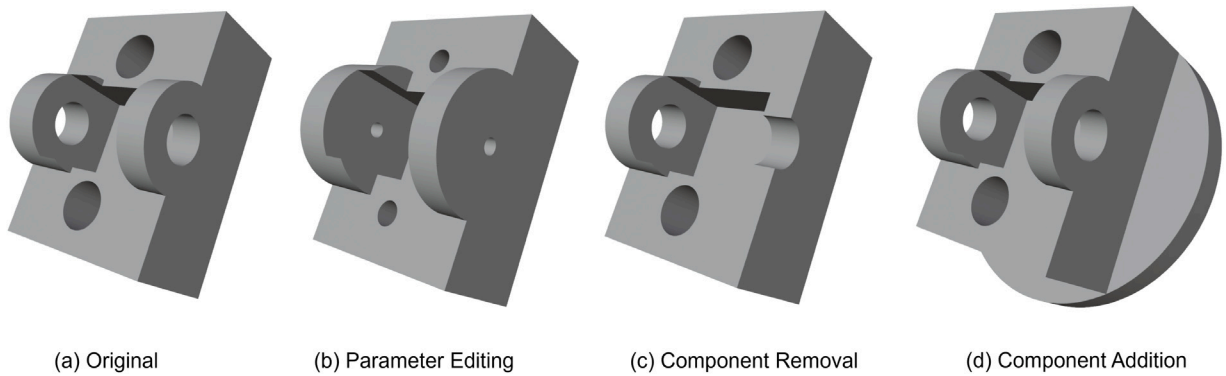


Fig. 8. Editing examples enabled by the learned CSG representation. From left to right: (a) original model, (b) parameter editing, (c) component removal, and (d) component addition. The edits are performed by modifying primitive parameters or manipulating subtrees in the CSG structure, enabling localized and interpretable changes without affecting unrelated parts of the model.

whereas descriptions that clearly specify both geometric parameters and part relationships lead to successful results. These examples illustrate that Text2CSG is sensitive to the level of geometric detail expressed in the input and benefits from more precise descriptions, which aligns with the interpretable and program-based nature of the generated CSG representations.

4.8. Discussion on text description precision

Fig. 10 analyzes the effect of text description precision on the generation behavior of Text2CSG. We consider textual descriptions with different levels of precision of the same target object, ranging from coarse semantic descriptions to more detailed and procedural ones. In the left example, the input text only specifies that the object is “a simple shape made of two cylinders”, This results in a hollow

cylindrical structure (i.e., a ring), which can be viewed as a larger cylinder subtracting a smaller one. Although this interpretation is not aligned with the intended design, it remains a valid and plausible CSG construction given the ambiguity of the input. In this case, the phrase “two cylinders” is satisfied by a subtraction-based configuration rather than a stacked or concatenated composition. As the description becomes more precise in the middle and right examples — explicitly specifying a wide base, a narrow extension, and their vertical relationship — the ambiguity is resolved. The decoder then transitions from a subtraction-based interpretation to a constructive composition of two cylinders with distinct roles, producing a solid model that better reflects the intended geometry. Overall, this example shows that the ring-shaped result arises from insufficient constraints in the textual description, rather than limitations of the decoding architecture. Explicit cues regarding relative scale, part hierarchy, and spatial arrangement

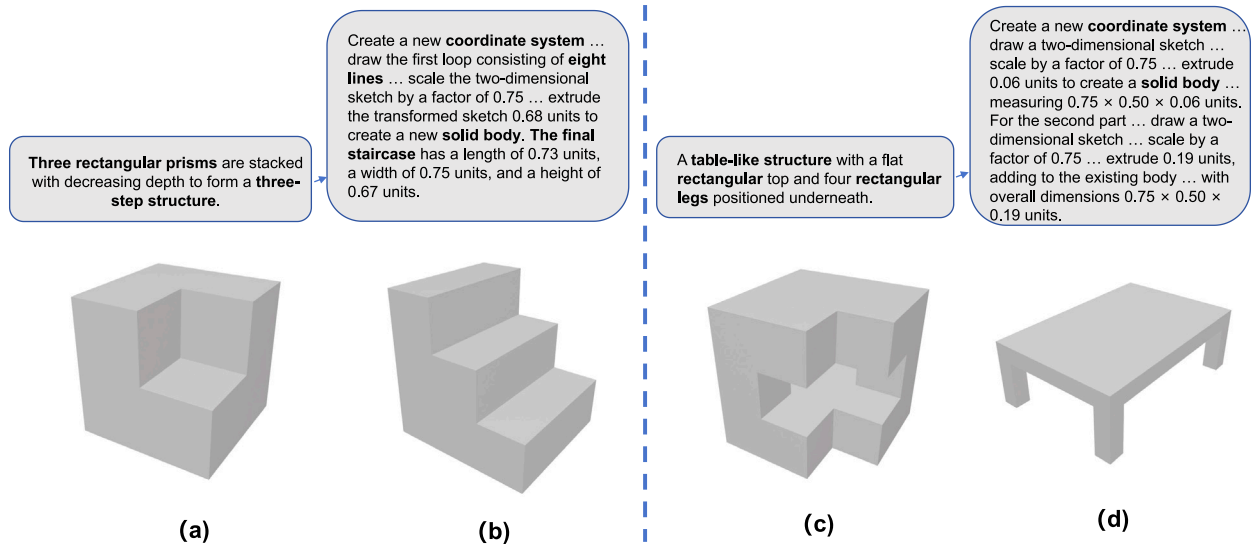


Fig. 9. Failed cases due to insufficient text descriptions. For the same target shape, brief or underspecified descriptions can lead to ambiguous interpretations and incomplete constructions. In (a) and (c), the input texts provide only coarse structural cues, resulting in outputs that do not fully match the intended three-step stair and table geometries. In contrast, when more detailed and explicit construction instructions are provided, as in (b) and (d), the model successfully recovers the desired shapes.

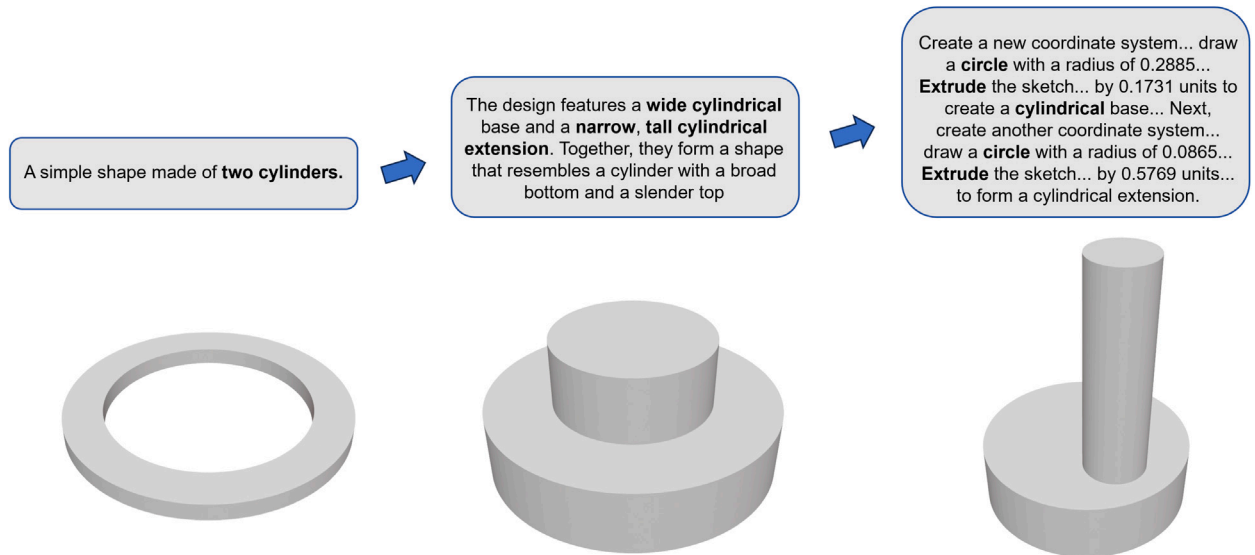


Fig. 10. Effect of text description precision on Text2CSG generation. Given the same object, increasingly detailed textual descriptions lead to progressively more structured and precise CSG programs and corresponding CAD geometries.

can help reduce ambiguity in Boolean operations for symbolic CAD generation.

5. Conclusion

We present Text2CSG, a two-stage framework for generating interpretable and structurally valid CAD models directly from natural-language descriptions. The framework separates geometric program learning from language grounding: a structured CSG decoder captures constructive logic, while a lightweight text-to-latent alignment module links high-level semantics to low-level geometry. To ensure reliable generation, we use a compact tokenization of CSG programs and include hierarchical supervision along with structure-aware loss terms that support syntactic correctness and compositional consistency. The structured nature of CSG representations facilitates alignment with semantic descriptions, thereby reducing the reliance on powerful pre-trained language models.

We construct a structured CSG dataset of about 25,000 models by processing and reorganizing existing CAD resources, specifically ABC [59] and Text2CAD [14]. Each model is paired with a natural-language description, enabling large-scale text-driven CAD generation. Experiments on this dataset demonstrate that Text2CSG produces accurate and reliable CSG programs, especially for multi-part designs where many existing approaches have difficulties. The structured and editable nature of the generated outputs suggests practical potential for real-world design workflows, where text-based interaction can speed up early-stage modeling and support quick changes in engineering and creative applications. In addition, the dataset, featuring clear Boolean hierarchies aligned with textual annotations, provides a foundation for future research on interpretable, text-driven CSG generation. Furthermore, our framework is encoder-agnostic and can be naturally extended to stronger language models (e.g., LLaMA3 [69], Qwen2 [70] or Mistral [71]), which we consider a promising direction for improving performance on more diverse and open-ended descriptions.

In addition, incorporating a prompt refinement stage using advanced language models may further improve the consistency and clarity of input descriptions. We leave a systematic study on prompt quality and its impact on text-to-CSG generation as future work.

CRedit authorship contribution statement

Luo Zhang: Writing – original draft, Visualization, Validation, Methodology, Investigation, Data curation, Conceptualization. **Gaochao Song:** Writing – original draft, Visualization, Validation, Resources, Methodology, Investigation, Data curation. **Haocong Rao:** Visualization, Validation, Methodology, Data curation. **Zhengyu Wen:** Writing – original draft, Validation, Methodology, Investigation. **Jiangbei Hu:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization. **Ying He:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This research was supported by the National Natural Science Foundation of China (No. 62402083), and Fundamental Research Funds for the Central Universities (DUT25RC(3)031).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.cad.2026.104127>.

Data availability

Data will be made available on request.

References

- [1] Cherng JG, Shao X-Y, Chen Y, Sferro PR. Feature-based part modeling and process planning for rapid response manufacturing. *Comput Ind Eng* 1998;34(2):515–30.
- [2] Elliott W. Interactive graphical CAD in mechanical engineering design. *Comput-Aided Des* 1978;10(2):91–100.
- [3] Feng N, Zhao H, Singh PK. Modeling in mechanical engineering CAD technology. *Comput-Aided Des Appl* 2022;19:1–14.
- [4] Szalabaj P. CAD principles for architectural design. Routledge; 2013.
- [5] Eastman C. Architectural CAD: A ten year assessment of the state of the art. *Comput-Aided Des* 1989;21(5):289–92.
- [6] Harrison DS, Newton AR, Spickelmier RL, Barnes TJ. Electronic CAD frameworks. *Proc IEEE* 1990;78(2):393–417.
- [7] Bérard A, Rizzi A, Isikveren A. CADac: A new geometry construction tool for aerospace vehicle pre-design and conceptual design. In: 26th AIAA applied aerodynamics conference. 2008, p. 6219.
- [8] Laidlaw DH, Trumbore WB, Hughes JF. Constructive solid geometry for polyhedral objects. In: Proceedings of the 13th annual conference on computer graphics and interactive techniques. 1986, p. 161–70.
- [9] Breen DE, Mauch S, Whitaker RT. 3D scan conversion of CSG models into distance volumes. In: Proceedings of the 1998 IEEE symposium on volume visualization. 1998, p. 7–14.
- [10] Sharma G, Goyal R, Liu D, Kalogerakis E, Maji S. CSGNet: Neural shape parser for constructive solid geometry. In: The IEEE conference on computer vision and pattern recognition. CVPR, 2018.
- [11] Yu F, Chen Z, Li M, Sanghi A, Shayani H, Mahdavi-Amiri A, Zhang H. CAPRI-Net: Learning compact CAD shapes with adaptive primitive assembly. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. CVPR, 2022, p. 11768–78.
- [12] Siddiqui Y, Alliegro A, Artemov A, Tommasi T, Sirigatti D, Rosov V, Dai A, Nießner M. MeshGPT: Generating triangle meshes with decoder-only transformers. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. CVPR, 2024.
- [13] Wu R, Xiao C, Zheng C. DeepCAD: A deep generative network for computer-aided design models. In: Proceedings of the IEEE/CVF international conference on computer vision. ICCV, 2021, p. 6772–82.
- [14] Khan MS, Sinha S, Sheikh TU, Stricker D, Ali SA, Afzal MZ. Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts. In: Advances in neural information processing systems. Vol. 37, 2024, p. 7552–79.
- [15] Govindarajan P, Baldelli D, Pathak J, Fournier Q, Chandar S. Cadmium: Fine-tuning code language models for text-driven sequential cad design. 2025, arXiv preprint arXiv:2507.09792.
- [16] Liao J, Xu J, Sun Y, Tang M, He S, Liao J, Yu S, Li Y, Guan X. Automated CAD modeling sequence generation from text descriptions via transformer-based large language models. In: Proceedings of the 63rd annual meeting of the association for computational linguistics. 2025, p. 21720–48.
- [17] Du Y, Chen S, Zan W, Li P, Wang M, Song D, Li B, Hu Y, Wang B. BlenderLLM: Training large language models for computer-aided design with self-improvement. 2024, arXiv:2412.14203.
- [18] Li J, Ma W, Li X, Lou Y, Zhou G, Zhou X. CAD-Llama: Leveraging large language models for computer-aided design parametric 3D model generation. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. CVPR, 2025, p. 18563–73.
- [19] Xu J, Zhao Z, Wang C, Liu W, Ma Y, Gao S. CAD-MLLM: Unifying multimodality-conditioned CAD generation with MLLM. 2024, arXiv:2411.04954.
- [20] Zhang Z, Sun S, Wang W, Cai D, Bian J. FlexCAD: Unified and versatile controllable CAD generation with fine-tuned large language models. In: International conference on learning representations. ICLR, 2025.
- [21] Mildenhall B, Srinivasan PP, Tancik M, Barron JT, Ramamoorthi R, Ng R. NeRF: Representing scenes as neural radiance fields for view synthesis. In: European conference on computer vision. ECCV, 2020.
- [22] Kerbl B, Kopanas G, Leimkühler T, Drettakis G. 3D Gaussian splatting for real-time radiance field rendering. *ACM Trans Graph* 2023;42(4):139:1–139:14.
- [23] Müller T, Evans A, Schied C, Keller A. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans Graph* 2022;41(4):102:1–102:15. <http://dx.doi.org/10.1145/3528223.3530127>.
- [24] Li J, Zhang L, Hu J, Zhang Z, Sun H, Song G, He Y. Real-time volume rendering with octree-based implicit surface representation. *Comput Aided Geom Design* 2024;111:102322. <http://dx.doi.org/10.1016/j.cagd.2024.102322>.
- [25] Li J, Wen Z, Zhang L, Hu J, Hou F, Zhang Z, He Y. GS-Octree: Octree-based 3D Gaussian splatting for robust object-level 3D reconstruction under strong lighting. In: Computer graphics forum. Vol. 43, Wiley Online Library; 2024, e15206.
- [26] Huang B, Yu Z, Chen A, Geiger A, Gao S. 2D Gaussian splatting for geometrically accurate radiance fields. In: SIGGRAPH 2024 conference papers. Association for Computing Machinery; 2024.
- [27] Lyu X, Sun Y-T, Huang Y-H, Wu X, Yang Z, Chen Y, Pang J, Qi X. 3DGSr: Implicit surface reconstruction with 3D Gaussian splatting. *ACM Trans Graph* 2024;43(6):1–12.
- [28] Xu J, Cheng W, Gao Y, Wang X, Gao S, Shan Y. InstantMesh: Efficient 3D mesh generation from a single image with sparse-view large reconstruction models. 2024, arXiv preprint arXiv:2404.07191.
- [29] Wang Z, Wang Y, Chen Y, Xiang C, Chen S, Yu D, Li C, Su H, Zhu J. CRM: Single image to 3D textured mesh with convolutional reconstruction model. In: European conference on computer vision. Springer; 2024, p. 57–74.
- [30] Tochilkin D, Pankratz D, Liu Z, Huang Z, Letts A, Li Y, Liang D, Laforte C, Jampani V, Cao Y-P. Triposr: Fast 3D object reconstruction from a single image. 2024, arXiv preprint arXiv:2403.02151.
- [31] Chen S, Chen X, Pang A, Zeng X, Cheng W, Fu Y, Yin F, Wang B, Yu J, Yu G, et al. MeshXL: Neural coordinate field for generative 3D foundation models. *Adv Neural Inf Process Syst* 2024;37:97141–66.
- [32] Liu M, Xu C, Jin H, Chen L, Varma T M, Xu Z, Su H. One-2-3-45: Any single image to 3D mesh in 45 seconds without per-shape optimization. *Adv Neural Inf Process Syst* 2024;36.
- [33] Tang J, Chen Z, Chen X, Wang T, Zeng G, Liu Z. Lgm: Large multi-view Gaussian model for high-resolution 3D content creation. In: European conference on computer vision. Springer; 2024, p. 1–18.
- [34] Chen Z, Wang Y, Wang F, Wang Z, Sun F, Liu H. V3D: Video diffusion models are effective 3D generators. *IEEE Trans Pattern Anal Mach Intell* 2025;(01):1–18. <http://dx.doi.org/10.1109/TPAMI.2025.3581312>.
- [35] Wang R, Yuan Y, Sun S, Bian J. Text-to-CAD generation through infusing visual feedback in large language models. In: International conference on machine learning. 2025.
- [36] Zhang Z, Liu K, Liu J, Wang W, Lin B, Xie L, Shen C, Cai D. GeoCAD: Local geometry-controllable CAD generation with large language models. 2025, arXiv:2506.10337.

- [37] Alrashedy K, Tambwekar P, Zaidi ZH, Langwasser M, Xu W, Gombolay M. Generating CAD code with vision-language models for 3D designs. In: The thirteenth international conference on learning representations. ICLR, 2025.
- [38] Zhang L, Le B, Akhtar N, Lam S-K, Ngo T. Large language models for computer-aided design: A survey. 2025, arXiv preprint arXiv:2505.08137.
- [39] Kapsalis T. CADgpt: Harnessing natural language processing for 3D modelling to enhance computer-aided design workflows. 2024, <http://dx.doi.org/10.48550/ARXIV.2401.05476>, CoRR abs/2401.05476, arXiv:2401.05476.
- [40] Badagabettu A, Yarlagadda SS, Farimani AB. Query2CAD: Generating CAD models using natural language queries. 2024, arXiv:2406.00144.
- [41] Xu X, Lambourne J, Jayaraman P, Wang Z, Willis K, Furukawa Y. BrepGen: A B-Rep generative diffusion model with structured latent geometry. ACM Trans Graph 2024;43(4):1–14.
- [42] Jayaraman PK, Lambourne JG, Desai N, Willis KD, Sanghi A, Morris NJ. SolidGen: An autoregressive model for direct B-Rep synthesis. In: The twelfth international conference on learning representations. ICLR, 2024.
- [43] Cherenkova K, Dupont E, Kacem A, Gusev G, Aouada D. SpelsNet: Surface primitive elements segmentation by B-Rep graph structure supervision. In: Globerson A, Mackey L, Belgrave D, Fan A, Paquet U, Tomczak J, Zhang C, editors. Advances in neural information processing systems. Vol. 37, Curran Associates, Inc.; 2024, p. 1251–69. <http://dx.doi.org/10.52202/079017-0039>.
- [44] Shen Z, Zhao M, ming Yan D, Wang W. Mesh2Brep: B-Rep reconstruction via robust primitive fitting and intersection-aware constraints. IEEE Trans Vis Comput Graphics 2025;31:6661–76, URL <https://api.semanticscholar.org/CorpusID:275379100>.
- [45] Yuan Y, Sun S, Liu Q, Bian J. CAD-editor: A locate-then-infill framework with automated training data synthesis for text-based CAD editing. 2025, arXiv:2502.03997. URL <https://arxiv.org/abs/2502.03997>.
- [46] Liu Y, Xu D, Yu X, Xu X, Cohen-Or D, Zhang H, Huang H. Hola: B-Rep generation using a holistic latent representation. ACM Trans Graph 2025;44(4):1–25.
- [47] You Y, Uy MA, Han J, Thomas R, Zhang H, You S, Guibas L. Img2CAD: Reverse engineering 3D CAD models from images through VLM-Assisted conditional factorization. In: SIGGRAPH Asia 2025 conference papers. New York, NY, USA: ACM; 2025.
- [48] Khan MS, Dupont E, Ali SA, Cherenkova K, Kacem A, Aouada D. CAD-SIGNet: CAD language inference from point clouds using layer-wise sketch instance guided attention. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. CVPR, 2024, p. 4713–22.
- [49] Xu X, Willis KD, Lambourne JG, Cheng C-Y, Jayaraman PK, Furukawa Y. SkexGen: Autoregressive generation of CAD construction sequences with disentangled codebooks. In: International conference on machine learning. ICML, PMLR; 2022, p. 24698–724.
- [50] Xu X, Jayaraman PK, Lambourne JG, Willis KD, Furukawa Y. Hierarchical neural coding for controllable CAD model generation. In: International conference on machine learning. 2023, p. 38443–61.
- [51] Ma W, Chen S, Lou Y, Li X, Zhou X. Draw step by step: Reconstructing CAD construction sequences from point clouds via multimodal diffusion. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. CVPR, 2024, p. 27154–63.
- [52] Zhang A, Jia W, Zou Q, Feng Y, Wei X, Zhang Y. Diffusion-CAD: Controllable diffusion model for generating computer-aided design models. IEEE Trans Vis Comput Graphics 2025.
- [53] Chen C, Wei J, Chen T, Zhang C, Yang X, Zhang S, Yang B, Foo C-S, Lin G, Huang Q, et al. CADrafter: Generating computer-aided design models from unconstrained images. In: Proceedings of the computer vision and pattern recognition conference. 2025, p. 11073–82.
- [54] Lee M, Zhang D, Jambon C, Kim YM. Brepdiff: Single-stage B-Rep diffusion model. In: Proceedings of the special interest group on computer graphics and interactive techniques conference papers. 2025, p. 1–11.
- [55] Yuan Z, Shi J, Huang Y. OpenECAD: An efficient visual language model for editable 3D-CAD design. Comput Graph 2024;124:104048. <http://dx.doi.org/10.1016/j.cag.2024.104048>.
- [56] Seff A, Ovadia Y, Zhou W, Adams RP. SketchGraphs: A large-scale dataset for modeling relational geometry in computer-aided design. In: ICML 2020 workshop on object-oriented learning. 2020.
- [57] Dupont E, Cherenkova K, Kacem A, Ali SA, Arzhannikov I, Gusev G, Aouada D. CADops-Net: Jointly learning CAD operation types and steps from boundary-representations. In: 2022 international conference on 3D vision. 3DV, Los Alamitos, CA, USA: IEEE Computer Society; 2022, p. 114–23. <http://dx.doi.org/10.1109/3DV57658.2022.00024>.
- [58] Zhou Q, Jacobson A. Thingi10K: A dataset of 10,000 3D-printing models. 2016, arXiv preprint arXiv:1605.04797.
- [59] Koch S, Matveev A, Jiang Z, Williams F, Artemov A, Burnaev E, Alexa M, Zorin D, Panozzo D. ABC: A big CAD model dataset for geometric deep learning. In: The IEEE conference on computer vision and pattern recognition. CVPR, 2019.
- [60] Willis KDD, Pu Y, Luo J, Chu H, Du T, Lambourne JG, Solar-Lezama A, Matusik W. Fusion 360 gallery: A dataset and environment for programmatic CAD construction from human design sequences. ACM Trans Graph 2021;40(4).
- [61] Los Alamos National Laboratory. MCNP® (Monte Carlo N-particle code). 2024, <https://mcnp.lanl.gov/>.
- [62] Ahmed M, Samee MR, Mercer RE. You only need attention to traverse trees. In: Korhonen A, Traum D, Màrquez L, editors. Proceedings of the 57th annual meeting of the association for computational linguistics. Florence, Italy: Association for Computational Linguistics; 2019, p. 316–22. <http://dx.doi.org/10.18653/v1/P19-1030>.
- [63] Li Z, Zhao H, Parnow K. Tree-structured attention with hierarchical accumulation. In: International conference on learning representations. ICLR, 2020.
- [64] Catalán J, Sauvan P, García J, Alguacil J, Ogando F, Sanz J. GEOUNED: A new conversion tool from CAD to Monte Carlo geometry. Nucl Eng Technol 2024.
- [65] Devlin J, Chang M-W, Lee K, Toutanova K. BERT: Pre-training of deep bidirectional transformers for language understanding. In: Burstein J, Doran C, Solorio T, editors. Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). Minneapolis, Minnesota: Association for Computational Linguistics; 2019, p. 4171–86. <http://dx.doi.org/10.18653/v1/N19-1423>, URL <https://aclanthology.org/N19-1423/>.
- [66] Li F, Zhang R, Zhang H, Zhang Y, Li B, Li W, Ma Z, Li C. LLaVA-NeXT-Interleave: Tackling multi-image, video, and 3D in large multimodal models. 2024, arXiv preprint arXiv:2407.07895.
- [67] Doris AC, Alam MF, Heyrani Nobari A, Ahmed F. CAD-Coder: An open-source vision-language model for Computer-Aided Design code generation. In: International design engineering technical conferences and computers and information in engineering conference. Vol. 89220, American Society of Mechanical Engineers; 2025, V03AT03A031.
- [68] Mews M, Aynedinov A, Schiller V, Eisert P, Akbik A. Don't mesh with me: Generating constructive solid geometry instead of meshes by fine-tuning a code-generation LLM. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops. CVPRW, 2025, AI for Content Creation Workshop.
- [69] Grattafiori A, Dubey A, Jauhri A, Pandey A, Kadian A, Al-Dahle A, Letman A, Mathur A, et al. The llama 3 herd of models. 2024, arXiv:2407.21783. URL <https://arxiv.org/abs/2407.21783>.
- [70] Yang A, Yang B, Hui B, Zheng B, Yu B, Zhou C, Li C, Li C, Liu D, Huang F, Dong G, Wei H, Lin H, Tang J, Wang J, Yang J, Tu J, Zhang J, Ma J, Yang J, Xu J, Zhou J, Bai J, He J, Lin J, Dang K, Lu K, Chen K, Yang K, Li M, Xue M, Ni N, Zhang P, Wang P, Peng R, Men R, Gao R, Lin R, Wang S, Bai S, Tan S, Zhu T, Li T, Liu T, Ge W, Deng X, Zhou X, Ren X, Zhang X, Wei X, Ren X, Liu X, Fan Y, Yao Y, Zhang Y, Wan Y, Chu Y, Liu Y, Cui Z, Zhang Z, Guo Z, Fan Z. Qwen2 technical report. 2024, arXiv:2407.10671. URL <https://arxiv.org/abs/2407.10671>.
- [71] Jiang AQ, Sablayrolles A, Mensch A, Bamford C, Chaplot DS, de las Casas D, Bressand F, Lengyel G, Lample G, Saulnier L, Lavaud LR, Lachaux M-A, Stock P, Scao TL, Lavril T, Wang T, Lacroix T, Sayed WE. Mistral 7B. 2023, arXiv:2310.06825. URL <https://arxiv.org/abs/2310.06825>.